

Experiment – 08 | Materialized Views in PostgreSQL

Name	Samridhi Singh	Section	24AIT_KRG-1_A
UID	24BAI70514	Semester	5th
Branch	BE-CSE(AIML)	Subject	Advanced Database Management System

Home Task

Home Task: Materialized Views in PostgreSQL

Hard Level – Create mv_category_sales using category, product, customer and order information. The report uses JOINS, GROUP BY, SUM, COUNT(DISTINCT), calculated values and CASE statements. Then demonstrate that the materialized view stays stale after new data is inserted until REFRESH MATERIALIZED VIEW is executed, and compare the original analytical query with the materialized-view query using EXPLAIN ANALYZE.

1. Create Source Tables and Insert Data

```
CREATE TABLE category_master (  
    category_id VARCHAR(10) PRIMARY KEY,  
    category_name VARCHAR(50)  
);  
  
CREATE TABLE product_master (  
    product_id VARCHAR(10) PRIMARY KEY,  
    product_name VARCHAR(100),  
    category_id VARCHAR(10) REFERENCES category_master(category_id)  
);  
  
CREATE TABLE customer_master (  
    customer_id VARCHAR(10) PRIMARY KEY,  
    customer_name VARCHAR(100)  
);  
  
CREATE TABLE order_details (  
    order_id VARCHAR(10) PRIMARY KEY,  
    customer_id VARCHAR(10) REFERENCES customer_master(customer_id),  
    product_id VARCHAR(10) REFERENCES product_master(product_id),  
    quantity INT,  
    unit_price NUMERIC(12,2)  
);  
  
INSERT INTO category_master VALUES  
('C1', 'Electronics'),  
('C2', 'Clothing'),  
('C3', 'Books');  
  
INSERT INTO product_master VALUES  
('P101', 'Laptop', 'C1'),  
('P102', 'Headphones', 'C1'),  
('P201', 'Jacket', 'C2'),  
('P202', 'Shoes', 'C2'),  
('P301', 'SQL Book', 'C3');  
  
INSERT INTO customer_master VALUES  
('U1', 'Aman'), ('U2', 'Priya'), ('U3', 'Rahul'), ('U4', 'Simran');
```

```

INSERT INTO order_details VALUES
('0101', 'U1', 'P101', 1, 5000),
('0102', 'U2', 'P102', 2, 1000),
('0103', 'U1', 'P201', 1, 3000),
('0104', 'U3', 'P202', 2, 2000),
('0105', 'U4', 'P301', 3, 500),
('0106', 'U2', 'P101', 1, 5000),
('0107', 'U3', 'P201', 1, 3000),
('0108', 'U4', 'P102', 1, 1000);

```

2. Original Analytical Query

```

SELECT
    c.category_id,
    c.category_name,
    SUM(od.quantity) AS total_products_sold,
    COUNT(DISTINCT od.order_id) AS total_orders,
    COUNT(DISTINCT od.customer_id) AS unique_customers,
    SUM(od.quantity * od.unit_price) AS total_revenue,
    ROUND(
        SUM(od.quantity * od.unit_price)::NUMERIC
        / NULLIF(COUNT(DISTINCT od.order_id), 0),
        2
    ) AS avg_revenue_per_order,
    CASE
        WHEN SUM(od.quantity * od.unit_price) >= 10000 THEN 'Excellent'
        WHEN SUM(od.quantity * od.unit_price) >= 6000 THEN 'Good'
        WHEN SUM(od.quantity * od.unit_price) >= 3000 THEN 'Average'
        ELSE 'Low'
    END AS category_performance
FROM category_master c
JOIN product_master p ON c.category_id = p.category_id
JOIN order_details od ON p.product_id = od.product_id
GROUP BY c.category_id, c.category_name
ORDER BY c.category_id;

```

3. Run EXPLAIN ANALYZE on the Original Query

```

EXPLAIN ANALYZE
SELECT
    c.category_id, c.category_name,
    SUM(od.quantity) AS total_products_sold,
    COUNT(DISTINCT od.order_id) AS total_orders,
    COUNT(DISTINCT od.customer_id) AS unique_customers,
    SUM(od.quantity * od.unit_price) AS total_revenue,
    ROUND(SUM(od.quantity * od.unit_price)::NUMERIC /
        NULLIF(COUNT(DISTINCT od.order_id), 0), 2) AS avg_revenue_per_order,
    CASE
        WHEN SUM(od.quantity * od.unit_price) >= 10000 THEN 'Excellent'
        WHEN SUM(od.quantity * od.unit_price) >= 6000 THEN 'Good'
        WHEN SUM(od.quantity * od.unit_price) >= 3000 THEN 'Average'
        ELSE 'Low'
    END AS category_performance
FROM category_master c
JOIN product_master p ON c.category_id = p.category_id
JOIN order_details od ON p.product_id = od.product_id
GROUP BY c.category_id, c.category_name
ORDER BY c.category_id;

```

4. Create the Materialized View

```

CREATE MATERIALIZED VIEW mv_category_sales AS
SELECT
    c.category_id,
    c.category_name,
    SUM(od.quantity) AS total_products_sold,
    COUNT(DISTINCT od.order_id) AS total_orders,
    COUNT(DISTINCT od.customer_id) AS unique_customers,
    SUM(od.quantity * od.unit_price) AS total_revenue,
    ROUND(SUM(od.quantity * od.unit_price)::NUMERIC /
        NULLIF(COUNT(DISTINCT od.order_id), 0), 2) AS avg_revenue_per_order,
    CASE
        WHEN SUM(od.quantity * od.unit_price) >= 10000 THEN 'Excellent'
        WHEN SUM(od.quantity * od.unit_price) >= 6000 THEN 'Good'
        WHEN SUM(od.quantity * od.unit_price) >= 3000 THEN 'Average'
        ELSE 'Low'
    END AS category_performance
FROM category_master c
JOIN product_master p ON c.category_id = p.category_id
JOIN order_details od ON p.product_id = od.product_id
GROUP BY c.category_id, c.category_name
ORDER BY c.category_id;

```

5. Display the Materialized View

```
SELECT * FROM mv_category_sales ORDER BY category_id;
```

Expected output

category_id	category_name	total_products_sold	total_orders	unique_customers	total_revenue	avg_revenue_per_order	category_performance
C1	Electronics	5	4	3	12000	3000.00	Excellent
C2	Clothing	4	3	2	10000	3333.33	Excellent
C3	Books	3	1	1	1500	1500.00	Low

6. Compare Query Performance

```
EXPLAIN ANALYZE
SELECT * FROM mv_category_sales ORDER BY category_id;
```

Compare the execution time reported by PostgreSQL with the original analytical query. The materialized view stores the query result, so the SELECT normally avoids repeating the complete JOIN, GROUP BY and aggregation work.

7. Demonstrate Stale Data

```
INSERT INTO order_details
VALUES ('0109', 'U1', 'P102', 1, 1000);

SELECT * FROM order_details WHERE order_id = '0109';

SELECT * FROM mv_category_sales
WHERE category_id = 'C1';
```

8. Refresh the Materialized View

```
REFRESH MATERIALIZED VIEW mv_category_sales;

SELECT * FROM mv_category_sales
ORDER BY category_id;
```

After the refresh, Electronics reflects the new order: total_products_sold = 6, total_orders = 5, unique_customers = 3, total_revenue = 13000 and avg_revenue_per_order = 2600.00. The category remains classified as Excellent under the stated business rules.

9. Performance Check After Refresh

```
EXPLAIN ANALYZE
SELECT * FROM mv_category_sales
ORDER BY category_id;
```

Business Rules Used

```
Total Products Sold = SUM(quantity)
Total Orders = COUNT(DISTINCT order_id)
Unique Customers = COUNT(DISTINCT customer_id)
Total Revenue = SUM(quantity × unit_price)
Average Revenue per Order = Total Revenue / Total Orders
```

```
Excellent: Revenue >= 10,000
Good:      Revenue >= 6,000 and < 10,000
Average:   Revenue >= 3,000 and < 6,000
Low:       Revenue < 3,000
```